

参考答案与解析

命题：大模型推理与部署课程组 校对：教研组

一、选择题答案

1. B	4. D	7. B	10. C
2. C	5. B	8. B	11. B
3. B	6. D	9. B	12. D

- 1. B** 注意力计算 QK^T 形状为 $n \times n$ ，每元素代价 d ，故为 $O(n^2d)$ ；再乘 V 同阶。A 是单层 attention 头内复杂度；C 是线性层复杂度；D 是 n, d 同阶时的极端估计。
- 2. C** Decode 阶段每步若不缓存 KV，需要重新计算历史所有 token 的 K, V ，复杂度 $O(n^2d)$ ；缓存后只需 $q \cdot K$ 与 $\text{softmax} \cdot V$ ，复杂度 $O(nd)$ 。
- 3. B** 单步 Decode 需要读取全部 KV Cache: $\text{MHA} = 2Lhd_h \cdot 2$ 字节, $\text{GQA} = 2Lh_{\text{KV}}d_h \cdot 2$ 字节, $\text{MQA} = 2L \cdot 1 \cdot d_h \cdot 2$ 字节。MHA(8MB 级) > GQA > MQA，三者显存与读取量同序。
- 4. D** RoPE 仅是在 Q、K 上做旋转矩阵乘法，复杂度仍为 $O(nd)$ ；它不改变注意力的 $O(n^2d)$ 复杂度结构。A、B、C 均为正确描述。
- 5. B** Prefill 阶段一次处理长 prompt，每 token 计算量 $O(nd)$ ，算术强度高 $\rightarrow$ Compute-bound；Decode 阶段每步只处理 1 个 token，但需读取全部 KV Cache，算术强度低 $\rightarrow$ Memory-bound。
- 6. D** PagedAttention 以固定大小 Block（如 16 token）组织 KV Cache，消除内部/外部碎片与预分配浪费；它并不直接优化算子融合效率。
- 7. B** Continuous Batching（又称 Iteration-level Batching）以 token 为单位调度，请求完成后立即让出位置给新请求；In-flight Batching 是 NVIDIA TensorRT-LLM 中的实现名称，包含 Continuous Batching + 多模态/多请求 in-flight 优化，并非完全等价。
- 8. B** GPTQ 基于 Hessian 二阶信息逐列量化权重；AWQ 通过识别显著权重通道并按缩放因子等效转移量化难度；SmoothQuant 通过 S 矩阵将激活离群点平滑到权重侧。三者均为 PTQ。
- 9. B** 激活值在少数通道（通常占 0.1% 1%）出现远大于其它通道的离群点（如 30x），若用 per-tensor 量化，整体 scale 被离群点拉大，正常通道精度损失严重。
- 10. C** Speculative Decoding 通过拒绝采样：以 $\min(1, p/q)$ 接受草稿 token；若拒绝，则按 $(p - q)^+$ 归一化后重新采样。可以严格证明输出分布等于 p ，对 Greedy 与 Top-p 均有效。
- 11. B** FlashAttention IO 复杂度为 $O(N^2d^2/M)$ ，其中 M 为 SRAM 大小。当 M 较大时可显著低于标准注意力的 $O(N^2 + Nd)$ HBM 访问量。
- 12. D** TP 不切分 LayerNorm、RMSNorm 等层（这些层在 TP 中被复制在每个 rank 上）；Embedding/LM Head 可按词表维度切分。因此“所有层都做切分”是错误的。

二、填空题答案

- 13.** $O(n^2d)$ ； $O(n^2)$ （注意力分数矩阵 $n \times n$ ）。
- 14.** 单条请求 KV Cache 大小 $= 2 \cdot L \cdot n \cdot h \cdot d_h \cdot 2\text{B} = 2 \times 32 \times 4096 \times 32 \times 128 \times 2 = 2^{36}$ 字节 = 64 GB。
- 15.** $h/h_{\text{KV}} = 40/8 = 5$ ； $h_{\text{KV}} = 1$ 时 GQA 退化为 MQA。

16. $V_p = \min \left\{ k \mid \sum_{i=1}^k p_i \geq p \right\}$ (即累积概率首次达到 p 的最小前缀集合) ; $p_i / \sum_{w_j \in V_p} p_j$ 。
17. $R_m = \text{diag} \left(\begin{pmatrix} \cos m\theta_0 & -\sin m\theta_0 \\ \sin m\theta_0 & \cos m\theta_0 \end{pmatrix}, \dots, \begin{pmatrix} \cos m\theta_{d/2-1} & -\sin m\theta_{d/2-1} \\ \sin m\theta_{d/2-1} & \cos m\theta_{d/2-1} \end{pmatrix} \right)$; $m = n$ 。
18. 期望接受数 $= \sum_{i=1}^k \alpha^i = \alpha \frac{1-\alpha^k}{1-\alpha}$ (或独立近似下 $k\alpha$) ; 由 $2 \approx 4\alpha$ 得 $\alpha \approx 0.50$; 若按几何级数 $\alpha(1-\alpha^4)/(1-\alpha) \geq 2$, 解得 $\alpha \gtrsim 0.54$, 取 $\boxed{\alpha \geq 0.54}$ 。

三、解答题答案

19. KV Cache 必要性推导 (10 分)

(1) 不使用 KV Cache 时 (4 分)

生成第 $n+1$ 个 token, 需要重新计算前 $n+1$ 个位置的 K, V :

$$K = X_{1:n+1}W_K, \quad V = X_{1:n+1}W_V, \quad X \in \mathbb{R}^{(n+1) \times d}.$$

两矩阵乘法 FLOPs 各为 $2(n+1) \cdot d \cdot d = 2(n+1)d^2$, 合计 $4(n+1)d^2$ 。注意力计算

$$S = QK^\top \in \mathbb{R}^{(n+1) \times (n+1)}, \quad O = \text{softmax}(S)V$$

的 FLOPs 为 $2(n+1)^2d + 2(n+1)^2d = 4(n+1)^2d$ 。因此单步总 FLOPs $\approx O(n^2d + nd^2)$, 对长序列 $n \gg d$ 时主导项为 $O(n^2d)$ 。

(2) 使用 KV Cache 时 (4 分)

只需计算第 $n+1$ 个位置的 $q_{n+1} = x_{n+1}W_Q$, FLOPs 为 $2d^2$; 而 K, V 已缓存, 只需读取。注意力计算:

$$s = q_{n+1}K^\top \in \mathbb{R}^{1 \times n}, \quad o = \text{softmax}(s)V \in \mathbb{R}^{1 \times d}.$$

FLOPs $= 2nd + 2nd = 4nd$, 即 $O(nd)$ 。相比 (1) 中的 $O(n^2d)$, 下降了 n 倍。这是 KV Cache 的核心价值: 把”对全部历史 token 重复算 K, V ”换成”只算当前 q ”, 把二次项降为一次项。

(3) 显存公式与带宽瓶颈 (2 分)

单条请求 KV Cache 显存:

$$M_{KV} = 2(K+V) \times L \times n \times h \times d_h \times \text{bytes_per_scalar}.$$

Decode 阶段每步 FLOPs 仅 $O(nd)$, 但要读取 $O(L \cdot n \cdot h \cdot d_h)$ 字节 KV Cache, 算术强度 (FLOPs/Byte) 约为 $1/d_h \sim 1/128$, 远低于 GPU 的 Roofline 转折点 (如 A100 约 100 FLOPs/Byte), 因此 Decode 是 **Memory-bound**, 显存带宽成为瓶颈。

20. RoPE 数学推导 (10 分)

(1) 内积仅依赖相对位置 (5 分)

对每一对维度 $(2i, 2i+1)$, 记 $q_i = (q_{2i}, q_{2i+1})^\top$, k_i 同理。则

$$\langle R_m q, R_n k \rangle = \sum_{i=0}^{d/2-1} q_i^\top R_{m,i}^\top R_{n,i} k_i.$$

注意到 $R_{m,i}^\top R_{n,i} = R_{n-m,i}$ (旋转矩阵的群性质: 旋转角相减), 即

$$R_{m,i}^\top R_{n,i} = \begin{pmatrix} \cos((n-m)\theta_i) & -\sin((n-m)\theta_i) \\ \sin((n-m)\theta_i) & \cos((n-m)\theta_i) \end{pmatrix} = R_{n-m,i}.$$

因此

$$\langle R_m q, R_n k \rangle = \sum_i q_i^\top R_{n-m, i} k_i,$$

只依赖 $m - n$ (注意 $n - m$ 与 $m - n$ 仅符号之差, $\cos$ 偶函数/ $\sin$ 奇函数)。这表明 RoPE 把”绝对位置编码”转化为”相对位置编码”, 无需在 attention 中显式注入位置偏置。

(2) 长度外推优势与 NTK-aware scaling (5 分)

绝对位置编码训练到 L_{train} 后, 超出该长度时模型从未见过该位置的编码, 外推性能急剧下降。RoPE 的相对性意味着只要相对距离的分布在训练范围内, 模型即可处理; 但 $m\theta_i$ 在 $m > L_{\text{train}}$ 时仍超出训练时的 θ 周期, 故直接外推也有损失。

NTK-aware scaling 修改频率基底:

$$\theta'_i = \text{base}^{-2i/d} \cdot \left(\frac{L_{\text{train}}}{L_{\text{train}}} \right)^{-2i/d} \cdot s^{-2i/d}, \quad s = \frac{L_{\text{new}}}{L_{\text{train}}}.$$

直觉: 高频分量 (小 i) 几乎不缩放, 保持训练时的精细相对位置感知; 低频分量 (大 i) 按 s 缩放, 使其周期被拉长到新长度 L_{new} 。这等价于”在频率空间做插值”, 比线性位置插值 (PI) 保留更多高频信息。

21. FlashAttention 原理 (10 分)

(1) 标准注意力的 HBM 访问 (3 分)

记 $Q, K, V \in \mathbb{R}^{n \times d}$, $S, P, O \in \mathbb{R}^{n \times n}$ 。标准实现:

- 读 Q, K : $2nd$ 字节; 写 S : n^2 字节;
- 读 S 计算 softmax: n^2 字节; 写 P : n^2 字节;
- 读 P, V : $n^2 + nd$ 字节; 写 O : nd 字节。

总 HBM 访问 $\approx O(n^2 + nd)$, 且当 $n \gg d$ 时由 n^2 项主导, 远高于 $O(nd)$ 的算术下界。

(2) Tiling 与 running softmax (5 分)

设 SRAM 大小 M , 取 block size $B_r = \Theta(M/d)$ (行块), $B_c = \Theta(M/d)$ (列块)。将 Q 按 B_r 分块 $Q_1, \dots, Q_{T_r}$; K, V 按 B_c 分块 $K_1, \dots, K_{T_c}$ 。

对每个 Q_i 块, 依次载入 K_j, V_j 到 SRAM, 计算局部 $S_{ij} = Q_i K_j^\top$ (在 SRAM 内完成 softmax), 更新输出 O_i 。关键在于 softmax 分母需要全局 max 与 $\sum$, 故采用 running statistic:

$$\begin{aligned} m_{ij}^{\text{new}} &= \max(m_i^{\text{old}}, \text{rowmax}(S_{ij})), \\ \ell_{ij}^{\text{new}} &= e^{m_i^{\text{old}} - m_{ij}^{\text{new}}} \ell_i^{\text{old}} + \text{rowsum}(e^{S_{ij} - m_{ij}^{\text{new}}}), \\ \tilde{O}_{ij}^{\text{new}} &= e^{m_i^{\text{old}} - m_{ij}^{\text{new}}} \tilde{O}_i^{\text{old}} + e^{S_{ij} - m_{ij}^{\text{new}}} V_j. \end{aligned}$$

最终 $O_i = \tilde{O}_i / \ell_i$ 。所有 S, P 中间量从不写回 HBM, 仅 O 写回。HBM 访问次数: Q 每块被 T_c 次访问, K, V 每块被 T_r 次访问, 总 $\approx O(N^2 d^2 / M)$ 。

(3) Recomputation 与反向 IO (2 分)

反向传播时, FlashAttention 不存储 S, P , 而是在反向时重新分块计算 $S_{ij} = Q_i K_j^\top$ 。代价: 额外 $O(N^2 d)$ FLOPs (相对前向的 $O(N^2 d)$ 增加一倍), 但 HBM 访问仍为 $O(N^2 d^2 / M)$, 避免存储 N^2 大小的中间矩阵。在 HBM 带宽受限的现代 GPU 上, 这是”以算代存”的典型优化。

22. SmoothQuant 等价变换 (10 分)

(1) 等价性推导 (4 分)

设 $S = \text{diag}(s_1, \dots, s_{d_{\text{in}}})$ 为可逆对角矩阵。则

$$\mathbf{X}\mathbf{W} = \mathbf{X} \underbrace{SS^{-1}}_{=I} \mathbf{W} = (\mathbf{X}S)(S^{-1}\mathbf{W}).$$

令 $\widehat{\mathbf{X}} = \mathbf{X}S$, $\widehat{\mathbf{W}} = S^{-1}\mathbf{W}$, 则 $\mathbf{Y} = \widehat{\mathbf{X}}\widehat{\mathbf{W}}$, 运算结果在浮点误差范围内严格相等。

(2) s_j 的取值与 α 含义 (3 分)

设 $c_j = \max |\mathbf{X}_{:,j}|$ (激活通道 j 的最大绝对值), $w_j = \max |\mathbf{W}_{j,:}|$ (权重行 j 的最大绝对值)。SmoothQuant 取

$$s_j = \frac{c_j^\alpha}{w_j^{1-\alpha}}, \quad \alpha \in [0, 1].$$

- $\alpha = 0$: $s_j = 1/w_j$, 等价于只对权重做缩放, 激活不变;
- $\alpha = 1$: $s_j = c_j$, 把激活的离群点完全迁移到权重侧, 激活变得”平坦”。

实践中 $\alpha \approx 0.5$ (如 OPT-175B 用 0.5) 能取得激活与权重精度之间的平衡。

(3) 离群点迁移机理 (3 分)

变换后 $\widehat{\mathbf{X}}_{:,j} = s_j \mathbf{X}_{:,j}$, 方差变为 $s_j^2 \cdot \text{Var}(\mathbf{X}_{:,j})$ 。由于 $s_j \propto c_j^\alpha / w_j^{1-\alpha}$, 离群通道 (c_j 大) 被 s_j 缩小 (当 $\alpha < 1$), 从而压低离群点幅度; 权重侧 $\widehat{\mathbf{W}}_{j,:} = \mathbf{W}_{j,:}/s_j$ 被相应放大, 但权重分布本就较平坦 (无离群点), 放大后仍落在量化可表示范围内。这种”把难度从激活转移到权重”的等价变换让两侧都能用 INT8 高精度量化, 从而支持 W8A8 全量化推理。

23. Speculative Decoding 数学保证 (10 分)

(1) 接受/拒绝规则 (4 分)

对第 i 个候选 token $\tilde{x}_i$, 设目标模型分布 $p = p_\phi(\cdot | x_{<i})$, 草稿模型分布 $q = q_\theta(\cdot | x_{<i})$ 。接受概率

$$r_i = \min\left(1, \frac{p(\tilde{x}_i)}{q(\tilde{x}_i)}\right).$$

以概率 r_i 接受 $\tilde{x}_i$; 若拒绝, 则按修正分布

$$p_{\text{resample}}(x) = \frac{\max(0, p(x) - q(x))}{\sum_{x'} \max(0, p(x') - q(x'))}$$

重新采样一个 token, 并停止本轮验证。

(2) 输出分布等于 p 的证明 (4 分)

只需证明 $P(\text{output} = x) = p(x)$ 。考虑单个位置:

$$\begin{aligned} P(\text{output} = x) &= \underbrace{P(\text{accept } x)}_{\text{草稿提出 } x \text{ 且接受}} + \underbrace{P(\text{reject and resample } x)}_{\text{拒绝某 token 后采样到 } x} \\ &= q(x) \cdot \min\left(1, \frac{p(x)}{q(x)}\right) + \sum_{x'} q(x') \left(1 - \min\left(1, \frac{p(x')}{q(x')}\right)\right) \cdot p_{\text{resample}}(x) \\ &= \min(q(x), p(x)) + \left[\sum_{x'} (q(x') - p(x'))^+ \right] \cdot p_{\text{resample}}(x). \end{aligned}$$

注意到 $\sum_{x'} (q(x') - p(x'))^+ = \sum_{x'} (p(x') - q(x'))^+ =: Z$ (因 $\sum q = \sum p = 1$), 且 $p_{\text{resample}}(x) = (p(x) - q(x))^+ / Z$ 。代入:

$$P(\text{output} = x) = \min(q(x), p(x)) + (p(x) - q(x))^+ = p(x).$$

即证。该结论对任意分布 p (含 Top-p 截断后的分布) 成立。

(3) 加速比与 k 的选择 (2 分)

单轮 Speculative Decoding 总延迟 $\approx t_d + t_v$ (草稿 k 步 + 验证 1 步); 期望产出 $\mathbb{E}[\text{tokens}] = \alpha(1 - \alpha^k)/(1 - \alpha)$ 。加速比

$$S(k) = \frac{\mathbb{E}[\text{tokens}] \cdot t_v}{t_d + t_v} = \frac{\alpha(1 - \alpha^k)}{(1 - \alpha)} \cdot \frac{t_v}{t_d + t_v}.$$

k 选择原则: k 增大 $\rightarrow$ 期望 token 增多但草稿延迟线性增加; 最优 k^* 满足 $\partial S / \partial k = 0$, 即 $\alpha^k(-\ln \alpha)(t_d + t_v) \approx t_v \cdot \alpha^k$. (边际产出等于边际成本)。工程上常根据在线 Acceptance Rate 动态调整 $k \in \{2, 3, 4, 8\}$ 。

24. 压轴题: 系统级综合优化 (10 分)**(1) Chunked Prefill 缓解延迟毛刺 (3 分)**

根因: 长 Prefill 请求 (如 32k tokens) 以单次大 batch 形式提交, GPU 需数十毫秒才能完成, 期间 Decode 请求被阻塞在调度队列中, 造成 P99 毛刺。这是典型的”长 Compute-bound 任务阻塞 Memory-bound 任务”问题。

Chunked Prefill 方案: 将长度 L 的 Prefill 切分为若干 chunk (如 512/1024 tokens), 每步只处理一个 chunk, 并将其与现有 Decode 请求**同 batch 调度**。具体协同:

- **Chunk 大小选择:** 取使得单 chunk 计算时间 $\leq$ Decode 步调度间隔 (约 10–20 ms), 通常 512–2048 tokens;
- **与 Continuous Batching 协同:** 调度器每 iter 优先保证 Decode 请求的 KV Cache 命中, 剩余预算分配给 Prefill chunk; 新到达的短请求可插队;
- **KV Cache 衔接:** chunk 之间共享 KV Cache, 每个 chunk 的输出 K, V 累加到 PagedAttention 的 block table 中。

预期效果: P99 毛刺从”阻塞 30 ms+”降到”恒定 5–10 ms”, 吞吐基本不损失。

(2) SM 占用率低的优化方向 (3 分)

根因: Decode 阶段 batch 内每条请求只算 1 个 token, GEMM 退化为 GEMV, 计算密度低; 同时不同请求长度差异导致 padding 浪费。

优化方向 (至少 3 项):

- **提高并发 (Max-Num-Seqs $\uparrow$):** 通过 PagedAttention 让更多请求共享同 batch, 使 GEMV 变 GEMV-batch $\rightarrow$ GEMM, SM 利用率上升;
- **GQA / MQA:** 减少 KV Cache 读取量, 让单步 Decode 从纯 Memory-bound 逐渐向 Compute-bound 靠拢, SM 持续有任务;
- **Chunked Prefill 同 batch:** 将 Compute-bound 的 Prefill chunk 与 Memory-bound 的 Decode 混合, 平衡算术强度, SM 同时被两类任务填满;
- **CUDA Graph / 算子融合:** 减少 kernel launch 开销, 使 SM 不被空隙浪费; 融合 RMSNorm+Residual, QKV projection、Rotary+Attention 等;
- **Prefix Caching:** 复用 System Prompt 的 KV, 减少冷启动的算子开销, 让有效 batch 提高。

(3) 长文本 OOM 的混合方案 (4 分)

设计思路: 将”显存—内存—SSD”组织为三级存储, KV Cache 按热度在三级间流动。

数据流:

1. **PagedAttention (GPU 显存):** 只保留当前 Decode 窗口与高频请求的 KV, 按 block (16 token) 分页;
2. **KV Cache Offloading (CPU 内存):** 长上下文中”久未访问”的 block 通过 PCIe 异步 prefetch/evict 到 CPU 内存 (FlexGen 思想);

3. **NVMe SSD 兜底**: 超长 ($>128k$) 任务的远端 block 落盘, CPU 内存作为缓存层;

4. **Prefix Caching**: System Prompt 与共享文档前缀在所有请求间复用, 避免重复计算与存储。

量化收益估算 (70B 模型, $L = 80$, $h = 64$, $d_h = 128$, fp16):

单 token KV 显存 $= 2 \cdot 80 \cdot 64 \cdot 128 \cdot 2B \approx 2.6MB/\text{token}$ 。128k 任务总 KV $\approx 128000 \times 2.6MB \approx 332GB$, 远超 80GB 单卡。

采用方案后:

- GPU 显存仅保留近 4k token: $\approx 10GB$;
- CPU 内存 (256GB) 存放中段 100k token: $\approx 260GB$;
- 剩余 24k token 落 NVMe: $\approx 60GB$;
- 若 10 条并发请求共享同一 System Prompt (4k token), Prefix Caching 节省 $9 \times 10GB = 90GB$ 显存;
- 综合后单卡可承载 128k 任务并发数从 0 提升到约 5-10, 吞吐量提升 $5\times$ 以上。

关键瓶颈在于 PCIe 带宽 (A100 PCIe 4.0 $\approx 64GB/s$ 单向), 需配合预取 (lookahead) 与异步流 (CUDA stream) 掩盖传输延迟, 使 Decode 计算与 KV 拷贝重叠。